PitchSense

Nathan Taing

Live Presentation

Below I have linked the video to my presentation: https://youtu.be/iq8gtlNJD_o

Overview

This project was broken up into two parts. The first part was to develop a computer vision system to track the motion and classify based on the extracted trajectory features. The second portion was to accelerate the computation time of the pitch tracking script using Bender's GPU resources. I focused on the motion tracking portion as the part I wanted to accelerate because image processing is highly demanding and takes up the majority of the computation time. This task, however, is highly repetitive, making it the most logical spot to implement parallel processing.

GPU Acceleration?

As mentioned above, we implemented GPU acceleration into the ball tracking portion of the software. The base tracking script implements image-processing techniques such as grayscale, frame differencing and masks cleanups. Grayscale conversion simplifies each frame by removing the color information, leaving only the light intensity of the pixel. Frame differencing compares consecutive frames with the value of each pixel in the same location as the previous frame. Mask cleanup works to remove noise and strengthen the detected baseball region. Within the CUDA code, we mainly focused on frame differencing and threshold based motion-mask generation.

These techniques are well suited for GPU acceleration as they are repeated across every pixel in each video frame. We decided each GPU thread would be responsible for one pixel. For example, during frame differencing, each thread compares the intensity value of a pixel in the current frame with the value of the same pixel in the previous frame. If the difference is big enough, we mark that pixel as "seeing motion."

By assigning thousands of pixels to thousands of GPU threads, the program can perform the image-processing techniques in parallel as opposed to processing each pixel sequentially on the CPU. This reduces the computational demand from the CPU and allows the CPU to focus on the higher-level tracking logic.

Implementation Details

The purpose of the main.cu and kernel.cu is for the motion detection stage to experience a decrease in computation time. I will note I was not able to get the .cu files run when utilizing the pipeline, which is written in Python. However, I was able to implement a main and a kernel which was compiled and run in Bender to experience the time reduction.

The code specifically focuses on creating a motion mask from two consecutive frames. Since frame differencing is required by every pixel through every frame, I figured this was the most important technique to focus on. The main handles the benchmarks, generating input frames, allocating memory, transferring data between the CPU and GPU, launching the kernel, and checking correctness. The main benchmarks the difference between CPU implementation and GPU implementation of frame differencing. This is particularly important because as the baseball moves across the screen, different pixels will change intensity levels, signifying the baseball is moving across that pixel represented location. This is what primarily drives our motion tracking algorithm.

Within main.cu there is a CPU version of the motion-mask algorithm that we use as the baseline. The CPU baseline calculates the index of the RGB values, compares the previous frame's RGB values to the current frame's RGB values, and adds the absolute differences from each channel to create a total motion score. If the motion score is greater than the threshold we set, then the pixel will be classified as detecting motion.

Since Bender had issues reading the actual video file, the CUDA version generates synthetic video frames to simulate reading the baseball. This helps create a repeatable test environment that allows both systems to be compared equally.

The main processing loop is as follows, for each frame, the program generates a new synthetic current frame. It then runs both the CPU and GPU versions of the motion-mask. We then time each system and see how long it took to process the information of one frame. Each system has its own time measurement, which is separate and will be used to compare the computation time at the end of the program. At the end of each frame iteration, the current frame becomes the previous frame and we generate a new synthetic frame to work on.

The final output details the average CPU time and the average GPU time. We then can use this information to move on and create conclusions. These conclusions will be talked about in the results section of the report. Overall, it was an interesting experience implementing a CUDA language tracker compared to Python, but it was necessary to compare our findings.

Within the kernel.cu the file contains the CUDA implementation of the motion mask. Each thread is responsible for processing one pixel. For each valid pixel, the kernel computes the one-dimensional pixel index then computes the RGB color index, it is the same as the one detailed in main.cu. The kernel reads the previous and current RGB values from global memory, computes the difference between the previous and current frame values, and applies the threshold score. As mentioned before, if the score is over the threshold, the pixel is marked as detecting motion. This is significant because the kernel is executing the same logic as the main, however this is on a per thread basis, meaning it is in parallel.

We kept block sizes to 16 x 16 as this was the primary size used in class. The grid size was calculated by the following formula:

$$gridSize.x = (width + blockSize.x - 1) / blockSize.x$$
$$gridSize.y = (height + blockSize.y - 1) / blockSize.y$$

This is for a 1920 by 1080 frame, which means there are about 2,073,600 pixels we need to account for. Since each thread processes one pixel, the GPU can launch work for over two million pixel operations per frame pair. This is much more efficient than the sequential CPU execution for this type of repetitive pixel operation.

Documentation on how to run code

The full documentation can be found in the README.md. However, basic code will be copy and pastable below:

- Pipeline:

  ```
  python .\pipeline.py --video "C:\Users\natei\Downloads\Capcut Folder\testpitch2.mov" --pitch-id "test001" --pitch-type "unknown" --pitcher-handedness "RHP" --roi 900 350 2300 1450 --start-frame 12 --end-frame 110 --manual-anchor-frames 12 18 24 30 36 42 48 54 60 69 78 86 95 105 110 --training-csv ".\compiled_pitch_features_pathshape_30samples.csv" --model ".\classifier_output_pathshape\best_pitch_classifier_pathshape.joblib"
  ```

- CUDA code:

  ```
  nvcc main.cu kernel.cu -o cuda_motion

  ./cuda_motion
  ```

Results

The final result was a moderately accurate classifier. The confidence was about 60 - 70%, but the final output percentages are significantly lower at 40 - 60%. This is still a positive sign as the classifier can recognize each pitch, however it is not the most confident in its answers. Running 3 tests on pitches it has never seen before yielded positive results, but the system would benefit from a bigger sample size. However, with the 30 sample pitches (10 fastballs, 10 curveballs, and 10 changeups), it is a solid foundation to build upon.

The CUDA implementation showed a significant performance improvement. According to the environment, the GPU kernel time is around 357x faster compared to the CPU. This makes sense because each thread operates on one pixel per frame compared to the CPU working on all

the pixels per frame. It is worth noting this measurement focuses on the motion-mask computation itself and does not represent the full Python tracking pipeline. Additionally, the pass at the bottom means both the output of the GPU and CPU was the same. This signifies that the system did not sacrifice accuracy for speed. It would be highly beneficial to implement the CUDA implementation into the original .py code to boost the computation time. However, we were able to demonstrate a moderately working classifier.

Problems

The main limitations were related to the tracking accuracy. The baseball moves very quickly and would often blend into white backgrounds. Initially, I used in-game pitches, however the tracker would get confused with the strike zone overlay and stop tracking the ball. To solve this issue, I switched to bullpen videos, which do not contain the overlay and present a more accurate path of the pitch.

My next issue was the quality of the video. The technique I implemented would often lock onto other objects that were not the baseball due to a shaky camera or the ball blending into the background. To solve this problem, I decided to use manual anchors to guide the ball. This would help place markers on how the ball would travel, and allow the tracker to more accurately follow the path of the ball.

An issue related to the CUDA code was Bender's lack of ability to open .mov files. To solve this issue, we generated synthetic frames to simulate a moving baseball. This allowed the CPU and the GPU version to be compared in a controlled environment. Since the frames were the same, we can directly compare the processing power of both. However, this approach does not fully capture the complexity of raw videos.

Another issue is CUDA only accelerates the frame differencing step. It is one of the many techniques implemented to try to minimize the noise within the raw footage. This stage is important because of the comparison of consecutive frames, but it is only one technique. Because of this, the CUDA results should be interpreted as a benchmark for one of the computationally expensive stages instead of the entire tracking pipeline.

The full system still depends on CPU-side operations and the accuracy of the OpenCV tracking logic. It is important to note that frame differencing is one of the most computationally time-consuming steps, meaning if we can reduce this stage, we can significantly boost the speed of the program.